SMART HOSTEL MANAGEMENT SYSTEM

Integrated OOPS and DBMS project report

Submitted by

SRIDHARAN S – 250701737
SHAKILEASH I – 250701681
SOUNDARARAJAN S – 250701726
SOMESWARAN R - 250701729

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)



November 2026

RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report titled **"SMART HOSTEL MANAGEMENT SYSTEM"** is the bonafide work of **SRIDHARAN S (250701737) , SHAKILEASH I (250701681) , SOUNDARARAJAN S (250701726) , SOMESWARAN R (250701723)**" who carried out the project work under my supervision.

SIGNATURE

Internal Faculty Name

Course Incharge

Object oriented programming system

Department name

SIGNATURE

Internal Faculty Name

Couse Incharge

Database management system

Department name

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

The Smart Hostel Management System is a Java and MySQL based application designed to organize common hostel administration activities in a centralized database. The application provides a menu-driven interface through which users can manage student records, hostel rooms, room allocations, fee payments and complaints. An admin login provides access to dashboard information and reports. The system uses JDBC to connect Java classes to the MySQL database named hostel_db. Student and room details are stored in database tables, while allocation, payment and complaint records are linked to students through student identifiers. Room allocation checks student and room validity, verifies available capacity, and uses a database transaction to commit or roll back the operation. The dashboard summarizes student, room, occupancy, payment and complaint information. The project aims to reduce manual record keeping, improve access to hostel information, support accurate room occupancy tracking and make fee and complaint records easier to retrieve. The report describes the system structure, modules, database relationships and implementation based on the supplied Java source code.

1. INTRODUCTION

Hostel administration involves maintaining student details, assigning rooms, monitoring room occupancy, recording fee payments and responding to student complaints. When these activities are handled manually or in separate records, retrieving accurate information and maintaining consistency can become difficult.

1.1 Problem Statement

The project addresses the need for a centralized application to maintain hostel-related records. It provides structured operations for student registration and search, room creation and updates, room allocation, fee payment recording, complaint tracking and administrative reporting. The system also needs to prevent room allocation when a room has reached its capacity.

1.2 Project Overview

The application is implemented in Java using a console-based, menu-driven interface. The Main class connects to MySQL and presents options for Student Management, Room Management, Room Allocation, Fee Payment, Complaints, Admin Access and Exit. The modules use JDBC classes such as Connection, PreparedStatement, Statement and ResultSet to execute database operations.

1.3 Purpose

• Centralize student, room, allocation, payment and complaint records. • Provide convenient add, view, search and update operations where implemented. • Track room capacity, occupied beds and available beds. • Maintain payment records with dates

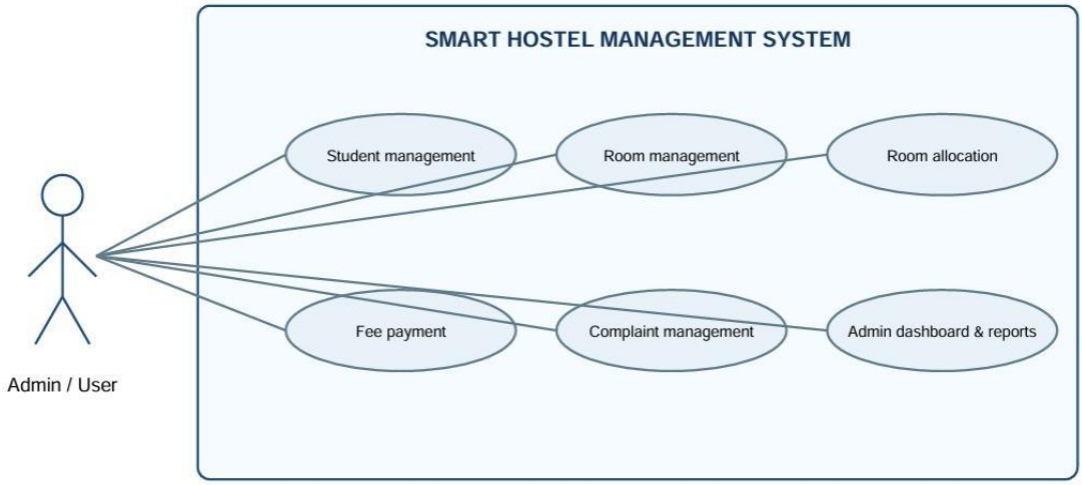
and payment status. • Allow administrators to review hostel statistics and reports. • Reduce repetitive manual record maintenance and improve data retrieval.

1.4 Scope

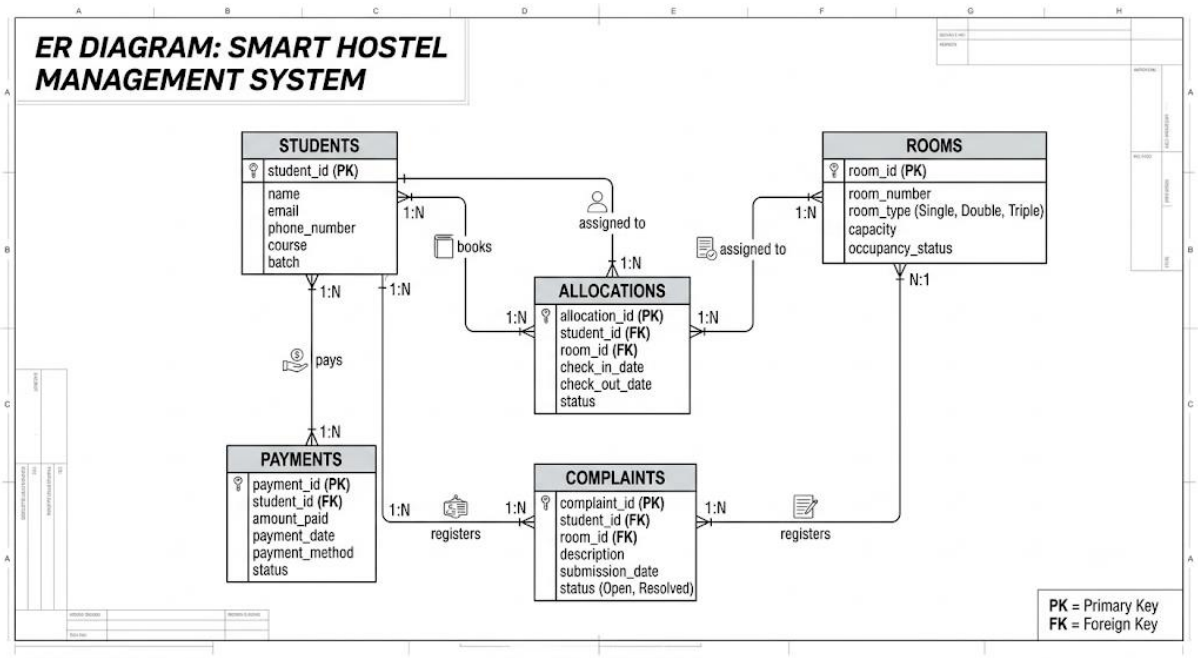
The current implementation covers student records (name, register number, department and phone), room records (room number, capacity and occupied count), room allocation records, fee payments, complaints and admin reports. The supplied code does not show features such as online payment gateway integration, student self-service accounts, attendance tracking, visitor management or automated notifications; these are outside the documented scope of this implementation.

3.3 Use Case Diagram

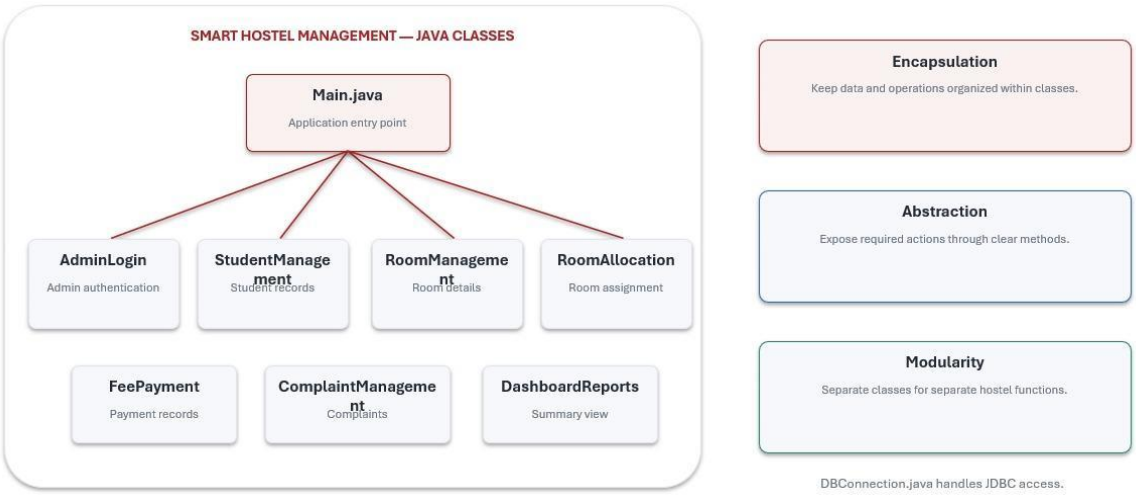
The Admin/User interacts with the system through the main menu. Admin authentication is required before accessing the dashboard and reports. The diagram summarizes the primary use cases implemented in the supplied classes.



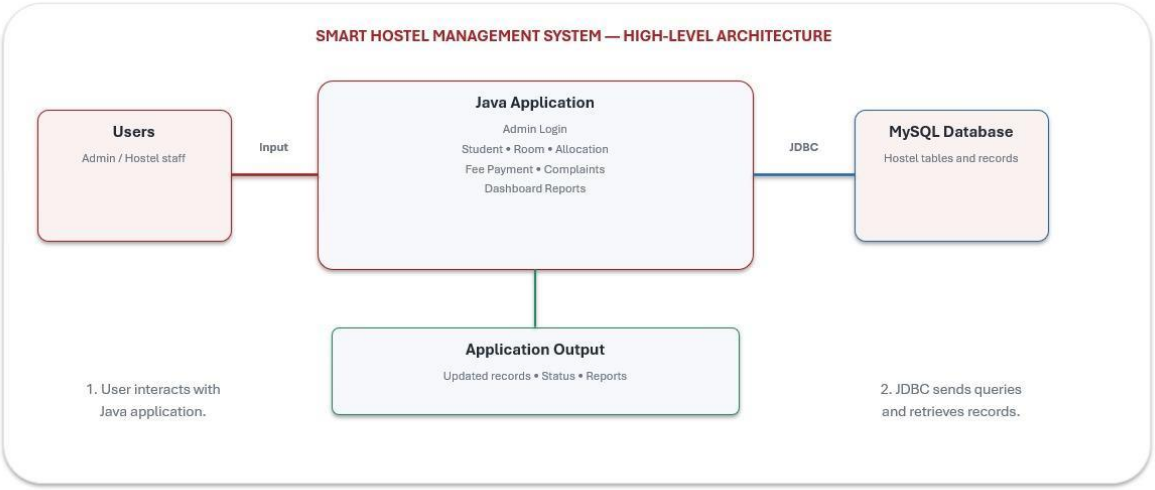
ENTITY RELATIONSHIP DIAGRAM



OOP Principles & Class Architecture



System Architecture Diagram



JAVA MAIN

```
import java.sql.Connection;
import java.sql.SQLException;
import java.util.Scanner;

public class Main {
   public static final String RESET = "\u001b[0m";
   public static final String CYAN = "\u001b[36m";
   public static final String YELLOW = "\u001b[33m";
   public static final String WHITE_BOLD = "\u001b[1;37m";
   public static final String GREEN = "\u001b[32m";
   public static final String RED = "\u001b[31m";
   public static final String BLUE = "\u001b[34m";
   public static final String PURPLE = "\u001b[35m";

   public Main() {
   }

   public static void main(String[] var0) {
      try {
         Connection var1 = DBConnection.getConnection();

         try {
            Scanner var2 = new Scanner(System.in);

            try {
               System.out.println("Database connected successfully!");
```

```
            int var3;
            do {

System.out.println("\u001b[36m┌──────────────────────────────────────────────────────────────────────────┐\u001b[0m");
                System.out.println("\u001b[36m│\u001b[1;37m       SMART HOSTEL MANAGEMENT SYSTEM         \u001b[36m│\u001b[0m");

System.out.println("\u001b[36m├──────────────────────────────────────────────────────────────────────────┤\u001b[0m");
                System.out.println("\u001b[36m│ \u001b[32m1. Student Management\u001b[36m                    │\u001b[0m");
                System.out.println("\u001b[36m│ \u001b[34m2. Room Management\u001b[36m                       │\u001b[0m");
                System.out.println("\u001b[36m│ \u001b[35m3. Room Allocation\u001b[36m                       │\u001b[0m");
                System.out.println("\u001b[36m│ \u001b[33m4. Fee Payment\u001b[36m                           │\u001b[0m");
                System.out.println("\u001b[36m│ \u001b[1;37m5. Complaints\u001b[36m                          │\u001b[0m");
                System.out.println("\u001b[36m│ \u001b[36m6. Admin Access\u001b[36m                          │\u001b[0m");

System.out.println("\u001b[36m├──────────────────────────────────────────────────────────────────────────┤\u001b[0m");
                System.out.println("\u001b[36m│ \u001b[31m7. Exit\u001b[36m                                  │\u001b[0m");

System.out.println("\u001b[36m└──────────────────────────────────────────────────────────────────────────┘\u001b[0m");
                System.out.print("\u001b[33m\ud83d\udc49 Enter your choice: \u001b[0m");
                var3 = var2.nextInt();
                switch (var3) {
                  case 1:
                    StudentManagement.manageStudents(var2);
                    break;
                  case 2:
                    RoomManagement.manageRooms(var2);
                    break;
                  case 3:
                    RoomAllocation.manageAllocations(var2);
                    break;
                  case 4:
                    FeePayment.managePayments(var2);
                    break;
                  case 5:
                    ComplaintManagement.manageComplaints(var2);
                    break;
```

```
                case 6:
                  if (AdminLogin.login(var2)) {
                    DashboardReports.manageDashboard(var2);
                  }
                  break;
                case 7:
                  System.out.println("Thank you!");
                  break;
                default:
                  System.out.println("Invalid choice. Try again.");
              }
            } while(var3 != 7);
          } catch (Throwable var7) {
            try {
              var2.close();
            } catch (Throwable var6) {
              var7.addSuppressed(var6);
            }

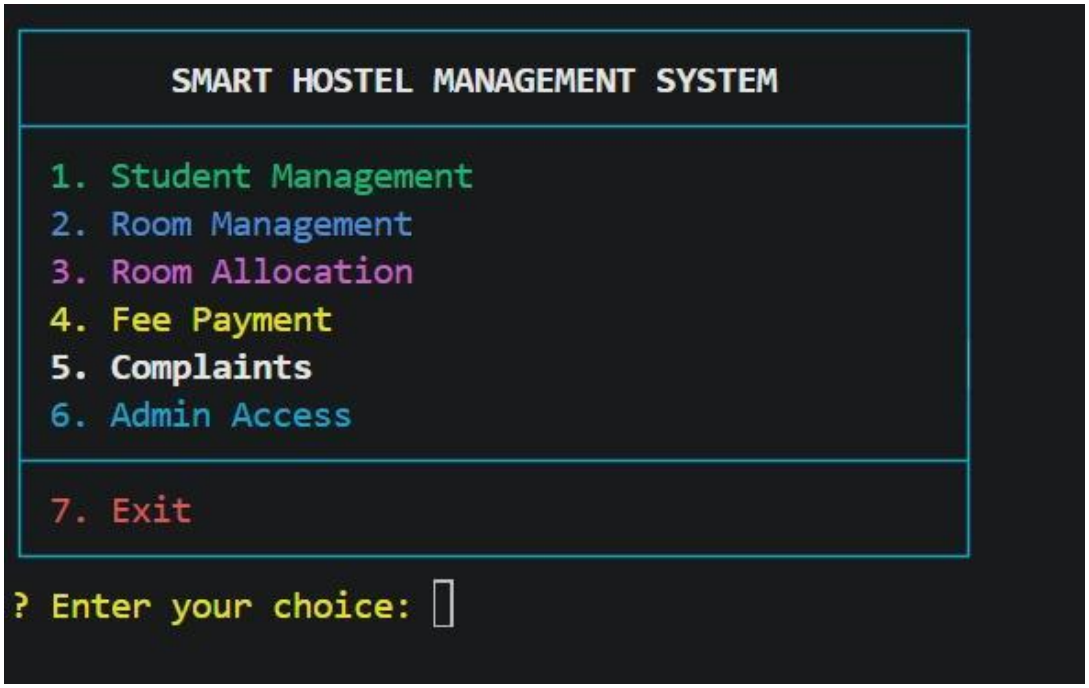
            throw var7;
          }

          var2.close();
        } catch (Throwable var8) {
          if (var1 != null) {
            try {
              var1.close();
            } catch (Throwable var5) {
              var8.addSuppressed(var5);
            }
          }

          throw var8;
        }

        if (var1 != null) {
          var1.close();
        }
      } catch (SQLException var9) {
        System.out.println("Database error: " + var9.getMessage());
      }

    }
}
```



STUDENT MANAGEMENT

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;

public class StudentManagement {
  public StudentManagement() {
  }

  public static void manageStudents(Scanner var0) {
    int var1;
    do {
      System.out.println("\n--- STUDENT MANAGEMENT ---");
      System.out.println("1. Add Student");
      System.out.println("2. View Students");
      System.out.println("3. Search Student");
      System.out.println("4. Back to Main Menu");
      System.out.print("Enter choice: ");
      var1 = var0.nextInt();
      var0.nextLine();
      switch (var1) {
        case 1:
          addStudent(var0);
```

```
            break;
         case 2:
            viewStudents();
            break;
         case 3:
            searchStudent(var0);
            break;
         case 4:
            System.out.println("Returning to main menu...");
            break;
         default:
            System.out.println("Invalid choice.");
      }
    } while(var1 != 4);

  }

  private static void addStudent(Scanner var0) {
    String var1 = "INSERT INTO students1 (name, register_no, department, phone)
VALUES (?, ?, ?, ?)";

    try {
      Connection var2 = DBConnection.getConnection();

      try {
        PreparedStatement var3 = var2.prepareStatement(var1);

        try {
          System.out.print("Enter student name: ");
          String var4 = var0.nextLine();
          System.out.print("Enter register number: ");
          String var5 = var0.nextLine();
          System.out.print("Enter department: ");
          String var6 = var0.nextLine();
          System.out.print("Enter phone number: ");
          String var7 = var0.nextLine();
          var3.setString(1, var4);
          var3.setString(2, var5);
          var3.setString(3, var6);
          var3.setString(4, var7);
          var3.executeUpdate();
          System.out.println("Student added successfully!");
        } catch (Throwable var10) {
          if (var3 != null) {
            try {
              var3.close();
            } catch (Throwable var9) {
              var10.addSuppressed(var9);
```

```
            }
         }

         throw var10;
      }

      if (var3 != null) {
         var3.close();
      }
   } catch (Throwable var11) {
      if (var2 != null) {
         try {
            var2.close();
         } catch (Throwable var8) {
            var11.addSuppressed(var8);
         }
      }

      throw var11;
   }

   if (var2 != null) {
      var2.close();
   }
} catch (SQLException var12) {
   System.out.println("Error adding student: " + var12.getMessage());
}

}

private static void viewStudents() {
   String var0 = "SELECT * FROM students1";

   try {
      Connection var1 = DBConnection.getConnection();

      try {
         Statement var2 = var1.createStatement();

         try {
            ResultSet var3 = var2.executeQuery(var0);

            try {
               System.out.println("\n--- STUDENT DETAILS ---");
               boolean var4 = false;

               while(var3.next()) {
                  var4 = true;
```

```
            System.out.println("Student ID: " + var3.getInt("student_id"));
            System.out.println("Name: " + var3.getString("name"));
            System.out.println("Register Number: " + var3.getString("register_no"));
            System.out.println("Department: " + var3.getString("department"));
            System.out.println("Phone: " + var3.getString("phone"));
            System.out.println("------------------------");
         }

         if (!var4) {
            System.out.println("No student records found.");
         }
      } catch (Throwable var9) {
         if (var3 != null) {
            try {
               var3.close();
            } catch (Throwable var8) {
               var9.addSuppressed(var8);
            }
         }

         throw var9;
      }

      if (var3 != null) {
         var3.close();
      }
   } catch (Throwable var10) {
      if (var2 != null) {
         try {
            var2.close();
         } catch (Throwable var7) {
            var10.addSuppressed(var7);
         }
      }

      throw var10;
   }

   if (var2 != null) {
      var2.close();
   }
} catch (Throwable var11) {
   if (var1 != null) {
      try {
         var1.close();
      } catch (Throwable var6) {
         var11.addSuppressed(var6);
      }
```

```
        }

        throw var11;
      }

      if (var1 != null) {
        var1.close();
      }
    } catch (SQLException var12) {
      System.out.println("Error viewing students: " + var12.getMessage());
    }

}

private static void searchStudent(Scanner var0) {
    System.out.print("Enter student register number: ");
    String var1 = var0.nextLine();
    String var2 = "SELECT * FROM students1 WHERE register_no = ?";

    try {
      Connection var3 = DBConnection.getConnection();

      try {
        PreparedStatement var4 = var3.prepareStatement(var2);

        try {
          var4.setString(1, var1);
          ResultSet var5 = var4.executeQuery();

          try {
            if (var5.next()) {
              System.out.println("Student ID: " + var5.getInt("student_id"));
              System.out.println("Name: " + var5.getString("name"));
              System.out.println("Register Number: " + var5.getString("register_no"));
              System.out.println("Department: " + var5.getString("department"));
              System.out.println("Phone: " + var5.getString("phone"));
            } else {
              System.out.println("Student not found.");
            }
          } catch (Throwable var11) {
            if (var5 != null) {
              try {
                var5.close();
              } catch (Throwable var10) {
                var11.addSuppressed(var10);
              }
            }
```

```
                throw var11;
            }

            if (var5 != null) {
               var5.close();
            }
         } catch (Throwable var12) {
            if (var4 != null) {
               try {
                  var4.close();
               } catch (Throwable var9) {
                  var12.addSuppressed(var9);
               }
            }

            throw var12;
         }

         if (var4 != null) {
            var4.close();
         }
      } catch (Throwable var13) {
         if (var3 != null) {
            try {
               var3.close();
            } catch (Throwable var8) {
               var13.addSuppressed(var8);
            }
         }

         throw var13;
      }

      if (var3 != null) {
         var3.close();
      }
   } catch (SQLException var14) {
      System.out.println("Error searching student: " + var14.getMessage());
   }

 }
}
```

```
--- STUDENT MANAGEMENT ---
1. Add Student
2. View Students
3. Search Student
4. Back to Main Menu
Enter choice: 2

--- STUDENT DETAILS ---
Student ID: 1
Name: SRI
Register Number: Rec2026001
Department: Cse
Phone: 9876543210
------------------------
Student ID: 2
Name: balaji
Register Number: rec2026002
Department: it
Phone: 9846373234
------------------------
```

ROOM MANAGEMENT

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;

public class RoomManagement {
  public RoomManagement() {
  }

  public static void manageRooms(Scanner var0) {
    int var1;
    do {
      System.out.println("\n--- ROOM MANAGEMENT ---");
      System.out.println("1. Add Room");
      System.out.println("2. View Rooms");
      System.out.println("3. Update Room Details");
      System.out.println("4. Back to Main Menu");
      System.out.print("Enter choice: ");
      var1 = var0.nextInt();
      var0.nextLine();
      switch (var1) {
```

```
            case 1:
               addRoom(var0);
               break;
            case 2:
               viewRooms();
               break;
            case 3:
               updateRoom(var0);
               break;
            case 4:
               System.out.println("Returning to main menu...");
               break;
            default:
               System.out.println("Invalid choice.");
         }
      } while(var1 != 4);

   }

   private static void addRoom(Scanner var0) {
      String var1 = "INSERT INTO rooms (room_number, capacity, occupied) VALUES (?, ?,
0)";

      try {
         Connection var2 = DBConnection.getConnection();

         label90: {
            try {
               PreparedStatement var3 = var2.prepareStatement(var1);

               label83: {
                  try {
                     System.out.print("Enter room number: ");
                     String var4 = var0.nextLine();
                     System.out.print("Enter room capacity: ");
                     int var5 = Integer.parseInt(var0.nextLine());
                     if (var5 > 0) {
                        var3.setString(1, var4);
                        var3.setInt(2, var5);
                        var3.executeUpdate();
                        System.out.println("Room added successfully!");
                        break label83;
                     }

                     System.out.println("Capacity must be greater than zero.");
                  } catch (Throwable var8) {
                     if (var3 != null) {
                        try {
```

```
                        var3.close();
                    } catch (Throwable var7) {
                        var8.addSuppressed(var7);
                    }
                }

                throw var8;
            }

            if (var3 != null) {
                var3.close();
            }
            break label90;
        }

        if (var3 != null) {
            var3.close();
        }
    } catch (Throwable var9) {
        if (var2 != null) {
            try {
                var2.close();
            } catch (Throwable var6) {
                var9.addSuppressed(var6);
            }
        }

        throw var9;
    }

    if (var2 != null) {
        var2.close();
    }

    return;
}

if (var2 != null) {
    var2.close();
}

} catch (NumberFormatException | SQLException var10) {
    System.out.println("Error adding room: " + ((Exception)var10).getMessage());
}
}

private static void viewRooms() {
    String var0 = "SELECT * FROM rooms ORDER BY room_number";
```

```
    try {
      Connection var1 = DBConnection.getConnection();

      try {
        Statement var2 = var1.createStatement();

        try {
          ResultSet var3 = var2.executeQuery(var0);

          try {
            System.out.println("\n--- ROOM DETAILS ---");
            System.out.printf("%-10s %-15s %-10s %-10s %-10s%n", "Room ID", "Room
Number", "Capacity", "Occupied", "Available");
            boolean var4 = false;

            while(var3.next()) {
              var4 = true;
              int var5 = var3.getInt("capacity");
              int var6 = var3.getInt("occupied");
              System.out.printf("%-10d %-15s %-10d %-10d %-10d%n",
var3.getInt("room_id"), var3.getString("room_number"), var5, var6, var5 - var6);
            }

            if (!var4) {
              System.out.println("No rooms found.");
            }
          } catch (Throwable var10) {
            if (var3 != null) {
              try {
                var3.close();
              } catch (Throwable var9) {
                var10.addSuppressed(var9);
              }
            }

            throw var10;
          }

          if (var3 != null) {
            var3.close();
          }
        } catch (Throwable var11) {
          if (var2 != null) {
            try {
              var2.close();
            } catch (Throwable var8) {
              var11.addSuppressed(var8);
```

```
            }
         }

         throw var11;
      }

      if (var2 != null) {
         var2.close();
      }
   } catch (Throwable var12) {
      if (var1 != null) {
         try {
            var1.close();
         } catch (Throwable var7) {
            var12.addSuppressed(var7);
         }
      }

      throw var12;
   }

   if (var1 != null) {
      var1.close();
   }
} catch (SQLException var13) {
   System.out.println("Error viewing rooms: " + var13.getMessage());
}

}

private static void updateRoom(Scanner var0) {
   System.out.print("Enter room number to update: ");
   String var1 = var0.nextLine();
   System.out.print("Enter new room number: ");
   String var2 = var0.nextLine();
   System.out.print("Enter new capacity: ");

   int var3;
   try {
      var3 = Integer.parseInt(var0.nextLine());
   } catch (NumberFormatException var11) {
      System.out.println("Please enter a valid number.");
      return;
   }

   if (var3 <= 0) {
      System.out.println("Capacity must be greater than zero.");
   } else {
```

```
        String var4 = "UPDATE rooms SET room_number = ?, capacity = ? WHERE
room_number = ? AND occupied <= ?";

        try {
           Connection var5 = DBConnection.getConnection();

           try {
              PreparedStatement var6 = var5.prepareStatement(var4);

              try {
                 var6.setString(1, var2);
                 var6.setInt(2, var3);
                 var6.setString(3, var1);
                 var6.setInt(4, var3);
                 int var7 = var6.executeUpdate();
                 if (var7 > 0) {
                    System.out.println("Room details updated successfully!");
                 } else {
                    System.out.println("Room not found, or capacity cannot be less than current
occupancy.");
                 }
              } catch (Throwable var12) {
                 if (var6 != null) {
                    try {
                       var6.close();
                    } catch (Throwable var10) {
                       var12.addSuppressed(var10);
                    }
                 }

                 throw var12;
              }

              if (var6 != null) {
                 var6.close();
              }
           } catch (Throwable var13) {
              if (var5 != null) {
                 try {
                    var5.close();
                 } catch (Throwable var9) {
                    var13.addSuppressed(var9);
                 }
              }

              throw var13;
           }
```

```
            if (var5 != null) {
               var5.close();
            }
         } catch (SQLException var14) {
            System.out.println("Error updating room: " + var14.getMessage());
         }

      }
   }
}
```

```
--- ROOM MANAGEMENT ---
1. Add Room
2. View Rooms
3. Update Room Details
4. Back to Main Menu
Enter choice: 2

--- ROOM DETAILS ---
Room ID    Room Number     Capacity   Occupied   Available
1                          150        1          149

--- ROOM MANAGEMENT ---
1. Add Room
2. View Rooms
3. Update Room Details
4. Back to Main Menu
Enter choice: 3
Enter room number to update:
Enter new room number: a101
Enter new capacity: 150
Room details updated successfully!
```

ROOM ALLOCATION

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;

public class RoomAllocation {
   public RoomAllocation() {
   }

   public static void manageAllocations(Scanner var0) {
      int var1;
      do {
```

```
        System.out.println("\n--- ROOM ALLOCATION ---");
        System.out.println("1. Allocate Room");
        System.out.println("2. View Allocations");
        System.out.println("3. Back to Main Menu");
        System.out.print("Enter choice: ");
        var1 = var0.nextInt();
        var0.nextLine();
        switch (var1) {
          case 1:
            allocateRoom(var0);
            break;
          case 2:
            viewAllocations();
            break;
          case 3:
            System.out.println("Returning to main menu...");
            break;
          default:
            System.out.println("Invalid choice.");
        }
      } while(var1 != 3);

    }

    private static void allocateRoom(Scanner var0) {
      System.out.print("Enter student ID: ");
      int var1 = var0.nextInt();
      var0.nextLine();
      System.out.print("Enter room ID: ");
      int var2 = var0.nextInt();
      var0.nextLine();
      String var3 = "SELECT capacity, occupied FROM rooms WHERE room_id = ?";
      String var4 = "SELECT student_id FROM students1 WHERE student_id = ?";
      String var5 = "INSERT INTO allocations (student_id, room_id, allocation_date) VALUES
(?, ?, CURDATE())";
      String var6 = "UPDATE rooms SET occupied = occupied + 1 WHERE room_id = ? AND
occupied < capacity";

      try {
        Connection var7 = DBConnection.getConnection();

        label510: {
          label511: {
            label531: {
              try {
                var7.setAutoCommit(false);

                try {
```

```
label512: {
   PreparedStatement var8 = var7.prepareStatement(var4);

   label513: {
      try {
         var8.setInt(1, var1);
         ResultSet var9 = var8.executeQuery();

         label489: {
            try {
               if (var9.next()) {
                  break label489;
               }

               System.out.println("Student ID not found.");
               var7.rollback();
            } catch (Throwable var41) {
               if (var9 != null) {
                  try {
                     var9.close();
                  } catch (Throwable var36) {
                     var41.addSuppressed(var36);
                  }
               }

               throw var41;
            }

            if (var9 != null) {
               var9.close();
            }
            break label513;
         }

         if (var9 != null) {
            var9.close();
         }
      } catch (Throwable var42) {
         if (var8 != null) {
            try {
               var8.close();
            } catch (Throwable var35) {
               var42.addSuppressed(var35);
            }
         }

         throw var42;
      }
```

```
if (var8 != null) {
   var8.close();
}

var8 = var7.prepareStatement(var3);

label515: {
   label516: {
      try {
         var8.setInt(1, var2);
         ResultSet var50 = var8.executeQuery();

         label473: {
            label472: {
               try {
                  if (var50.next()) {
                     int var10 = var50.getInt("capacity");
                     int var11 = var50.getInt("occupied");
                     if (var11 < var10) {
                        break label473;
                     }

                     System.out.println("Room is full.");
                     var7.rollback();
                     break label472;
                  }

                  System.out.println("Room ID not found.");
                  var7.rollback();
               } catch (Throwable var39) {
                  if (var50 != null) {
                     try {
                        var50.close();
                     } catch (Throwable var34) {
                        var39.addSuppressed(var34);
                     }
                  }

                  throw var39;
               }

               if (var50 != null) {
                  var50.close();
               }
               break label516;
            }
```

```
      if (var50 != null) {
         var50.close();
      }
      break label515;
   }

   if (var50 != null) {
      var50.close();
   }
} catch (Throwable var40) {
   if (var8 != null) {
      try {
         var8.close();
      } catch (Throwable var33) {
         var40.addSuppressed(var33);
      }
   }

   throw var40;
}

if (var8 != null) {
   var8.close();
}

var8 = var7.prepareStatement(var5);

try {
   var8.setInt(1, var1);
   var8.setInt(2, var2);
   var8.executeUpdate();
} catch (Throwable var37) {
   if (var8 != null) {
      try {
         var8.close();
      } catch (Throwable var32) {
         var37.addSuppressed(var32);
      }
   }

   throw var37;
}

if (var8 != null) {
   var8.close();
}

var8 = var7.prepareStatement(var6);
```

```
                        try {
                           var8.setInt(1, var2);
                           if (var8.executeUpdate() == 0) {
                              throw new SQLException("Room capacity changed. Please try
again.");
                           }
                        } catch (Throwable var38) {
                           if (var8 != null) {
                              try {
                                 var8.close();
                              } catch (Throwable var31) {
                                 var38.addSuppressed(var31);
                              }
                           }

                           throw var38;
                        }

                        if (var8 != null) {
                           var8.close();
                        }

                        var7.commit();
                        System.out.println("Room allocated successfully!");
                        break label512;
                     }

                     if (var8 != null) {
                        var8.close();
                     }
                     break label511;
                  }

                  if (var8 != null) {
                     var8.close();
                  }
                  break label510;
               }

               if (var8 != null) {
                  var8.close();
               }
               break label531;
            }
         } catch (SQLException var43) {
            var7.rollback();
            if (var43.getErrorCode() == 1062) {
```

```
                    System.out.println("This student already has a room allocation.");
                  } else {
                    System.out.println("Allocation failed: " + var43.getMessage());
                  }
                } finally {
                  var7.setAutoCommit(true);
                }
              } catch (Throwable var45) {
                if (var7 != null) {
                  try {
                    var7.close();
                  } catch (Throwable var30) {
                    var45.addSuppressed(var30);
                  }
                }

                throw var45;
              }

              if (var7 != null) {
                var7.close();
              }

              return;
            }

            if (var7 != null) {
              var7.close();
            }

            return;
          }

          if (var7 != null) {
            var7.close();
          }

          return;
        }

        if (var7 != null) {
          var7.close();
        }

      } catch (SQLException var46) {
        System.out.println("Database error: " + var46.getMessage());
      }
    }
```

```
   private static void viewAllocations() {
      String var0 = "SELECT a.allocation_id, s.name, s.register_no, r.room_number,
a.allocation_date FROM allocations a JOIN students1 s ON a.student_id = s.student_id
JOIN rooms r ON a.room_id = r.room_id ORDER BY a.allocation_id";

      try {
         Connection var1 = DBConnection.getConnection();

         try {
            Statement var2 = var1.createStatement();

            try {
               ResultSet var3 = var2.executeQuery(var0);

               try {
                  System.out.println("\n--- ALLOCATION DETAILS ---");
                  System.out.printf("%-15s %-20s %-20s %-15s %-15s%n", "Allocation ID",
"Student Name", "Register No", "Room Number", "Date");
                  boolean var4 = false;

                  while(var3.next()) {
                     var4 = true;
                     System.out.printf("%-15d %-20s %-20s %-15s %-15s%n",
var3.getInt("allocation_id"), var3.getString("name"), var3.getString("register_no"),
var3.getString("room_number"), var3.getDate("allocation_date"));
                  }

                  if (!var4) {
                     System.out.println("No allocations found.");
                  }
               } catch (Throwable var9) {
                  if (var3 != null) {
                     try {
                        var3.close();
                     } catch (Throwable var8) {
                        var9.addSuppressed(var8);
                     }
                  }

                  throw var9;
               }

               if (var3 != null) {
                  var3.close();
               }
            } catch (Throwable var10) {
               if (var2 != null) {
```

```
            try {
               var2.close();
            } catch (Throwable var7) {
               var10.addSuppressed(var7);
            }
         }

         throw var10;
      }

      if (var2 != null) {
         var2.close();
      }
   } catch (Throwable var11) {
      if (var1 != null) {
         try {
            var1.close();
         } catch (Throwable var6) {
            var11.addSuppressed(var6);
         }
      }

      throw var11;
   }

   if (var1 != null) {
      var1.close();
   }
} catch (SQLException var12) {
   System.out.println("Error viewing allocations: " + var12.getMessage());
}

  }
}
```

```
--- ROOM ALLOCATION ---
1. Allocate Room
2. View Allocations
3. Back to Main Menu
Enter choice: 2

--- ALLOCATION DETAILS ---
Allocation ID   Student Name         Register No          Room Number     Date
1               SRI                  Rec2026001           a101            2026-10-10

    ROOM ALLOCATION
```

FEE PAYMENT

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;

public class FeePayment {
   public FeePayment() {
   }

   public static void managePayments(Scanner var0) {
      int var1;
      do {
         System.out.println("\n--- FEE PAYMENT ---");
         System.out.println("1. Record Payment");
         System.out.println("2. View All Payments");
         System.out.println("3. View Student Payments");
         System.out.println("4. Back to Main Menu");
         System.out.print("Enter choice: ");
         var1 = var0.nextInt();
         var0.nextLine();
         switch (var1) {
            case 1:
               recordPayment(var0);
               break;
            case 2:
               viewAllPayments();
               break;
            case 3:
               viewStudentPayments(var0);
               break;
            case 4:
               System.out.println("Returning to main menu...");
               break;
            default:
               System.out.println("Invalid choice.");
         }
      } while(var1 != 4);

   }

   private static void recordPayment(Scanner var0) {
      System.out.print("Enter student ID: ");
      int var1 = var0.nextInt();
      var0.nextLine();
      System.out.print("Enter payment amount: ");
      double var2 = var0.nextDouble();
```

```
      var0.nextLine();
      if (var2 <= (double)0.0F) {
         System.out.println("Amount must be greater than zero.");
      } else {
         String var4 = "SELECT student_id FROM students1 WHERE student_id = ?";
         String var5 = "INSERT INTO payments (student_id, amount, payment_date, status)
VALUES (?, ?, CURDATE(), 'PAID')";

         try {
            Connection var6 = DBConnection.getConnection();

            label142: {
               try {
                  PreparedStatement var7;
                  label132: {
                     var7 = var6.prepareStatement(var4);

                     try {
                        var7.setInt(1, var1);
                        ResultSet var8 = var7.executeQuery();

                        label106: {
                           try {
                              if (!var8.next()) {
                                 System.out.println("Student ID not found.");
                                 break label106;
                              }
                           } catch (Throwable var16) {
                              if (var8 != null) {
                                 try {
                                    var8.close();
                                 } catch (Throwable var14) {
                                    var16.addSuppressed(var14);
                                 }
                              }

                              throw var16;
                           }

                           if (var8 != null) {
                              var8.close();
                           }
                           break label132;
                        }

                        if (var8 != null) {
                           var8.close();
                        }
```

```
      } catch (Throwable var17) {
         if (var7 != null) {
            try {
               var7.close();
            } catch (Throwable var13) {
               var17.addSuppressed(var13);
            }
         }

         throw var17;
      }

      if (var7 != null) {
         var7.close();
      }
      break label142;
   }

   if (var7 != null) {
      var7.close();
   }

   var7 = var6.prepareStatement(var5);

   try {
      var7.setInt(1, var1);
      var7.setDouble(2, var2);
      var7.executeUpdate();
      System.out.println("Payment recorded successfully!");
   } catch (Throwable var15) {
      if (var7 != null) {
         try {
            var7.close();
         } catch (Throwable var12) {
            var15.addSuppressed(var12);
         }
      }

      throw var15;
   }

   if (var7 != null) {
      var7.close();
   }
} catch (Throwable var18) {
   if (var6 != null) {
      try {
         var6.close();
```

```
                } catch (Throwable var11) {
                    var18.addSuppressed(var11);
                }
            }

            throw var18;
        }

        if (var6 != null) {
            var6.close();
        }

        return;
    }

    if (var6 != null) {
        var6.close();
    }

  } catch (SQLException var19) {
    System.out.println("Payment error: " + var19.getMessage());
  }
 }
}

 private static void viewAllPayments() {
  String var0 = "SELECT p.payment_id, s.name, s.register_no, p.amount,
p.payment_date, p.status FROM payments p JOIN students1 s ON p.student_id =
s.student_id ORDER BY p.payment_id";

  try {
    Connection var1 = DBConnection.getConnection();

    try {
      Statement var2 = var1.createStatement();

      try {
        ResultSet var3 = var2.executeQuery(var0);

        try {
          System.out.println("\n--- ALL PAYMENTS ---");
          System.out.printf("%-12s %-20s %-18s %-12s %-15s %-12s%n", "Payment ID",
"Student Name", "Register No", "Amount", "Payment Date", "Status");
          boolean var4 = false;

          while(var3.next()) {
            var4 = true;
```

```
                System.out.printf("%-12d %-20s %-18s %-12.2f %-15s %-12s%n",
var3.getInt("payment_id"), var3.getString("name"), var3.getString("register_no"),
var3.getDouble("amount"), var3.getDate("payment_date"), var3.getString("status"));
             }

             if (!var4) {
                System.out.println("No payments found.");
             }
          } catch (Throwable var9) {
             if (var3 != null) {
                try {
                   var3.close();
                } catch (Throwable var8) {
                   var9.addSuppressed(var8);
                }
             }

             throw var9;
          }

          if (var3 != null) {
             var3.close();
          }
       } catch (Throwable var10) {
          if (var2 != null) {
             try {
                var2.close();
             } catch (Throwable var7) {
                var10.addSuppressed(var7);
             }
          }

          throw var10;
       }

       if (var2 != null) {
          var2.close();
       }
    } catch (Throwable var11) {
       if (var1 != null) {
          try {
             var1.close();
          } catch (Throwable var6) {
             var11.addSuppressed(var6);
          }
       }

       throw var11;
```

```
        }

        if (var1 != null) {
          var1.close();
        }
      } catch (SQLException var12) {
        System.out.println("Error viewing payments: " + var12.getMessage());
      }

    }

    private static void viewStudentPayments(Scanner var0) {
      System.out.print("Enter student ID: ");
      int var1 = var0.nextInt();
      var0.nextLine();
      String var2 = "SELECT payment_id, amount, payment_date, status FROM payments
WHERE student_id = ? ORDER BY payment_date";

      try {
        Connection var3 = DBConnection.getConnection();

        try {
          PreparedStatement var4 = var3.prepareStatement(var2);

          try {
            var4.setInt(1, var1);
            ResultSet var5 = var4.executeQuery();

            try {
              System.out.println("\n--- STUDENT PAYMENTS ---");
              boolean var6 = false;

              while(var5.next()) {
                var6 = true;
                System.out.printf("Payment ID: %d | Amount: %.2f | Date: %s | Status: %s%n",
var5.getInt("payment_id"), var5.getDouble("amount"), var5.getDate("payment_date"),
var5.getString("status"));
              }

              if (!var6) {
                System.out.println("No payments found for this student.");
              }
            } catch (Throwable var11) {
              if (var5 != null) {
                try {
                  var5.close();
                } catch (Throwable var10) {
                  var11.addSuppressed(var10);
```

```
                }
            }

            throw var11;
        }

        if (var5 != null) {
            var5.close();
        }
    } catch (Throwable var12) {
        if (var4 != null) {
            try {
                var4.close();
            } catch (Throwable var9) {
                var12.addSuppressed(var9);
            }
        }

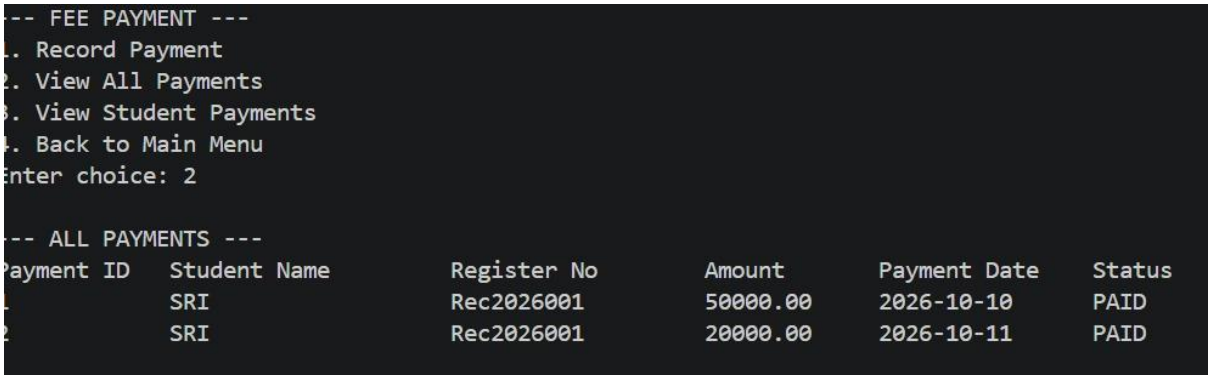
        throw var12;
    }

    if (var4 != null) {
        var4.close();
    }
} catch (Throwable var13) {
    if (var3 != null) {
        try {
            var3.close();
        } catch (Throwable var8) {
            var13.addSuppressed(var8);
        }
    }

    throw var13;
}

if (var3 != null) {
    var3.close();
}
} catch (SQLException var14) {
    System.out.println("Error viewing student payments: " + var14.getMessage());
}

}
}
```



CONCLUSION

The Smart Hostel Management System provides a centralized Java and MySQL based approach for managing important hostel operations. The implemented modules support student registration and search, room management, room allocation, fee payment recording, complaint handling and administrative reporting. JDBC enables the application to store and retrieve information from the MySQL database, while validation and transaction handling support more reliable operations, particularly during room allocation. The project demonstrates the integration of object-oriented Java programming with relational database operations. Future improvements may include secure password hashing, role-based access, a graphical user interface, room deallocation, payment receipts, online payment integration, notifications and automated backups.